Summer Internship Report

DESIGN AND IMPLEMENTATION OF POSIT AND RISC-V PROCESSOR

A report submitted in partial fulfilment of the requirements for

SUMMER INTERNSHIP

by

Mathew James

(Roll No. 240102061)

Under the supervision of

Prof. Roy Paily Palathinkal

roypaily@iitg.ac.in

from 12th of May to 20th of June

Department of Electrical and Electronics Engineering

Indian Institute of Technology Guwahati

Guwahati-781039, Assam, India, June-2025

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my supervisor, **Prof. Roy Paily Palathinkal**, for his unwavering support, insightful guidance, and constant encouragement through the course of this internship. His expertise and patience have been instrumental in shaping the direction and depth of my work.

I would like to extend my sincere gratitude to my mentor Anupam Kumari and Josephine S (Lab Technical In-charge) for their valuable suggestions, technical input, and continuous motivation.

Sincerely

MATHEW JAMES

(Roll No: 240102061)

ABSTRACT

The purpose of this internship was to impart a basic understanding and knowledge of digital electronics and computer architecture. The internship also enabled us to get a deeper understanding of floating-point numbers (especially the IEEE-754 single precision float), and the novel POSIT format used to approximate real numbers in hardware. We used Verilog HDL to design combinational and sequential circuits which were then simulated and synthesized on Xilinx Vivado. The generated bitstream was then used to run a FPGA (Basys Master 3 board). While working on POSIT format, we were able to understand its advantages in representing a wider range of information. We also realized its drawbacks for doing operations in a naïve implementation as it necessitated the use of a custom floating-point representation internally. This got us to the conclusion that a custom floating point which is appropriate to the usage scenario of the processor might have better performance. We wrote a 3 stage, pipelined RISC-V 32 base ISA processor which helped us understand the advantages of pipelining and RISC-V architecture. This gave us a grasp on solving pipelining hazards. The RISC-V processor had the following parameters: Area: 25471.800284 μm^2, Power: 1.0051 mW, and Delay: 2.96 ns

TABLE OF CONTENTS

INTRODUCTION

Digital electronics is a branch of electronics that deals with circuits, devices, and systems that use digital signals. Digital systems work with discrete signals, often represented as binary (0s and 1s).

Digital Logic Design is the process of designing circuits using simple logic gates to implement desired functions. These gates can be combined in various ways to create more complex digital circuits. The two main types of logic circuits are combinational logic and sequential logic.
Combination Logic Circuits: In combinational logic, the outputs only depend on the current inputs.
Sequential Logic Circuits: In sequential logic, the outputs are dependent on previous inputs and therefore memory elements that store previous states is required.

In Assignment 1, we tackle combination logic circuits including a 4:2 encoder, 2:4 decoder, 8:1 Multiplexer, 1:8 Demultiplexer, half adder, Full adder, 4-bit ripple carry adder and Multiplier

In Assignment 2, we tackle sequential logic circuits including a Mod-10 Asynchronous counter, Mod-10 Synchronous counter, Mod-10 up-down counter, 8-bit SISO, 8-bit PIPO, 8-bit SIPO, 8-bit PISO, Sequence detector (overlapping), and Sequence detector (non-overlapping).

RISC-V is an open standard instruction set architecture (ISA) based on established reduced instruction set computer (RISC) principles.

In Assignment 3, we designed a 3 stage, pipelined RISC-V 32 base ISA processor in Verilog.

POSITs are a new format used to represent real numbers in computing which is proposed as an alternative to the IEEE-754 floating point format. We implemented an IEEE 754 single precision float to POSIT (16,2) converter.

ASSIGNMENT-1

1. 4:2 encoder
   An encoder is a one-hot to binary converter. That is, if there are 2^n input lines, and at most only one of them will ever be high, the binary code of this 'hot' line is produced on the *n*-bit output lines. Here, we use 4 input lines with 2-bit output lines.
2. 2:4 decoder
   A binary decoder has n-output bits. It asserts exactly one of its n output bits, or none of them, for every integer input value. The "address" (bit number) of the activated output is specified by the integer input value. Here, we use a decoder with 4 output bits and 2 input bits to specify the address of the output bit.
3. 8:1 Multiplexer
   A multiplexer is a device that selects between several input signals and forwards the selected input to a single output line. The selection is directed by a separate set of digital inputs known as select lines. A multiplexer of 2^n inputs have n select lines, which are used to select the input line to select to the output line. Here, we have 8 inputs.
4. 1:8 Demultiplexer
   It is the converse of a multiplexer. A demultiplexer is a device that takes a single input signal and selectively forwards it to one of several output lines. Here we have 8 outputs.
5. Half adder
   The half adder adds two single binary digits A and B. It has two outputs, sum (S) and carry (C). The carry signal represents an overflow into the next digit of a multi-digit addition.
6. Full adder
   A full adder adds binary numbers and accounts for values carried in as well as out. A one-bit full-adder adds three one-bit numbers, often written as A, B, and C_{in}. A and B are the operands, and C_{in} is a bit carried in from the previous less-significant stage.
7. 4-bit ripple carry adder
   It is possible to create a logical circuit using multiple full adders to add N-bit numbers. Each full adder inputs a C_{in}, which is the C_{out} of the previous adder.
8. Multiplier
   The multiplication is done by calculating partial products (which are 0 or the first number), shifting them left, and then adding them together.

	LUT	Flip-Flops	Power	Delay
4:2 encoder	1	0	0.544	6.758
2:4 decoder	2	0	0.993	6.911
8:1 Mux	2	0	0.636	7.351
1:8 Demux	4	0	0.92	7.354
Half adder	1	0	0.735	6.764
Full adder	1	0	0.974	6.928
4-bit ripple carry adder	4	0	2.711	7.749
Multiplier	15	0	4.12	9.703

ASSIGNMENT-2

1. Mod-10 Asynchronous counter
   An Asynchronous Counter is a type of counter where each flip-flop is triggered by the output of the previous one, not by a common clock signal. In a mod-10 counter, the counter resets after every 10 counts.
2. Mod-10 Synchronous counter
   A Synchronous Counter is a type of counter where all the flip-flops are controlled by a single common clock signal. This ensures that all flip-flops in the counter change their states simultaneously, leading to synchronized counting. In a mod-10 counter, the counter resets after every 10 counts.
3. Mod-10 up-down counter
   An up/down counter, also known as a bidirectional counter, is a digital circuit that can count both up and down based on an input control signal. In a mod-10 counter, the counter resets after every 10 counts.
4. 8-bit SISO
   The shift register, which allows serial and produces a serial output is known as a Serial-In Serial-Out shift register. Since there is only one output, the data leaves the shift register one bit at a time in a serial pattern. The circuit consists of eight D flip-flops which are connected in a serial manner.
5. 8-bit PIPO
   The shift register, which allows parallel input and produces a parallel output, is known as Parallel-In Parallel-Out shift register. The circuit consists of eight D flip-flops which are connected. The clear signal and clock signals are connected to all eight flip-flops. In this type of register, there are no interconnections between the individual flip-flops since no serial shifting of the data is required.
6. 8-bit SIPO
   The shift register, which allows serial input and produces a parallel output, is known as the Serial-In Parallel-Out shift register. The circuit consists of eight D flip-flops which are connected. The clear signal is connected in addition to the clock signal to all eight flip flops to RESET them.
7. 8-bit PISO
   The shift register, which allows parallel input and produces a serial output is known as a Parallel-In Serial-Out shift register. The circuit consists of eight D flip-flops. The input data is connected individually to each flip-flop through a multiplexer at the input of every flip-flop. The output of the previous flip-flop and parallel data input are connected to the input of the MUX and the output of MUX is connected to the next flip-flop.
8. Sequence detector (overlapping)
   A sequence detector is the digital circuit that detects some input signal sequences from a set of binary data. In an overlapping sequence detector, the final bits of the sequence can be the start of another sequence.
9. Sequence detector (non-overlapping)
   Once sequence detection is completed, another sequence detection can be started without any overlap.

	LUT	Flip-Flops	Power	Delay
Mod-10 Asynchronous counter	5	4	0.074	7.054
Mod-10 Synchronous counter	6	4	0.074	7.12
Mod-10 up-down counter	4	5	0.074	1.742
8-bit SISO	0	8	0.071	1.682
8-bit PIPO	7	15	0.073	1.816
8-bit SIPO	0	15	0.073	1.632
8-bit PISO	7	8	0.071	1.761
Sequence detector(overlapping)	1	6	0.071	1.697
Sequence detector(Non-overlapping)	50	38	0.071	1.747

ASSIGNMENT-3

RISC-V is an open standard instruction set architecture (ISA) based on established reduced instruction set computer (RISC) principles.

We designed a 3 stage, pipelined RISC-V 32 base ISA processor in Verilog.

The processor included the following modules:

1. Stage 1 (Fetch): pc (program counter), and instr_r (instruction register).
2. Stage 2 (Decode): instr_decode (instruction decoder), reg_bank (register bank), control (control unit), reg_block (register block for pipelining), and store (store unit).
3. Stage 3 (Execute and Writeback): load (load unit), alu (ALU), and writeback (to writeback to register).

The processor stalls when the output of the previous instruction is used as input to the current instruction to allow for it to be written back to the register.

We used GNU RISCV toolchain to compile programs for the processor to run.

Processor Parameters:

1. Area: 25471.800284 μm^2
2. Power: 1.0051 mW
3. Delay: 2.96 ns

POSIT

POSITs are a new format used to represent real numbers in computing which is proposed as an alternative to the IEEE-754 floating point format.

POSIT is determined by its exponent length(es) and total length(n).
It is made of four parts: sign bit, regime, exponent, and mantissa.
1. Sign bit – The sign bit is used to indicate sign of the number, POSITs use 2's complement notation for negative numbers.
2. Regime – Regime is made up of either consecutive 1s or 0s which is usually terminated by a dedicated bit unless the regime extends the entirety of the length. The number of 1s or 0s indicates the scale factor to be used depending on the set exponent length.
3. Exponent – The length of the exponent depends on the type of POSIT we intend to use.
4. Mantissa – The length of mantissa is the leftover bits after the other parts.

The value of a POSIT is given by the formula:
$sign(p) \times useed^{k} \times 2^{2} \times (1 + f)$ where $useed = 2^{2^{es}}$

We implemented an IEEE 754 single precision float to POSIT (16,2) converter.

It takes the last two bits of the exponent of the IEEE float after removing the biasing as the exponent of the POSIT and adds the necessary number of 1s or 0s depending on the rest of the exponent bits. Mantissa is then truncated and stored in the rest of the bits of the bits. The 2's complement of the result is taken based on the sign bit of the float which is also used as the sign bit of the POSIT.

While working on POSITs, we realized that to make naïve implementations of general operations (like Addition, Multiplication, etc.) on POSITs, we convert it into a custom float of higher bit. Therefore, when compared with such a custom float, we had to use more logic to implement a POSIT operation. This is unfavorable. The custom float also had fixed mantissa width and therefore had better precision for all ranges, the disadvantage being that it takes more bits. The custom float required to represent a POSIT (16, 2) uses 19 bits.

Therefore, we concluded that custom floats that fit to the requirements of the intent of the processors may be better suited than a POSIT to improve performance.

CONCLUSION

From the above results, I have concluded that POSITs have interesting characteristics especially in storing a wider range of data. At the same time, the deconstruction of the data in a POSIT adds undesirable hardware costs when using it for replacing the current float extensions. Using custom floats which are designed according to the data being handled can be a more optimal approach.

This work has helped to give me an understanding of RISC V ISA. As a result, I am interested in the workings of the Rocket core RISC V processor. I also want to continue to improve the current processor that I have written using the techniques I will be seeing in the future in other processors so that it can be more full-fledged.